

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО»
(Университет ИТМО)**

Факультет программной инженерии и компьютерных технологий

Лабораторная работа №4

Вариант № 386131

Выполнил: Федоров Д. А.

Р3206

Проверил: Кулинич Я. В.

г. Санкт-Петербург 2026 г.

Оглавление

Текст задания:	2
Ход работы:	2
Задание 1.	2
Задание 2.	6
Задание 3.	8
Задание 4.	10
Вывод по лабораторной работе:	18

Текст задания:

Вариант 386131

Внимание! У разных вариантов разный текст задания!

1. Для своей программы из лабораторной работы #3 по дисциплине "Веб-программирование" реализовать:

- MBean, считающий общее число установленных пользователем точек, а также число точек, не попадающих в область. В случае, если пользователь совершил 4 "промаха" подряд, разработанный MBean должен отправлять оповещение об этом событии.
- MBean, определяющий процентное отношение "попаданий" к общему числу кликов пользователя по координатной плоскости.

2. С помощью утилиты JConsole провести мониторинг программы:

- Снять показания MBean-классов, разработанных в ходе выполнения задания 1.
- Определить имя и версию ОС, под управлением которой работает JVM.

3. С помощью утилиты VisualVM провести мониторинг и профилирование программы:

- Снять график изменения показаний MBean-классов, разработанных в ходе выполнения задания 1, с течением времени.
- Определить имя потока, потребляющего наибольший процент времени CPU.

4. С помощью утилиты VisualVM и профилировщика IDE NetBeans, Eclipse или Idea локализовать и устранить проблемы с производительностью в программе. По результатам локализации и устранения проблемы необходимо составить отчёт, в котором должна содержаться следующая информация:

- Описание выявленной проблемы.
- Описание путей устранения выявленной проблемы.
- Подробное (со скриншотами) описание алгоритма действий, который позволил выявить и локализовать проблему.

Студент должен обеспечить возможность воспроизведения процесса поиска и локализации проблемы по требованию преподавателя.

Рис 1. – текст задания.

Отчёт по работе должен содержать:

1. Текст задания.
2. Исходный код разработанных MBean-классов и сопутствующих классов.
3. Скриншоты программы JConsole со снятыми показаниями, выводы по результатам мониторинга.
4. Скриншоты программы VisualVM со снятыми показаниями, выводы по результатам профилирования.
5. Скриншоты программы VisualVM с комментариями по ходу поиска утечки памяти.
6. Выводы по работе.

Рис 2. – текст задания.

Ход работы:

Задание 1.

Исходный код разработанных mBeans:

HitPercentageMBean:

```
package mBeans;

public interface HitPercentageMBean { 1 usage 1 implementation
    double getHitPercentage(); no usages 1 implementation
}
```

Рис 3. – HitPercentageMBean.

HitPercentage:

```
1 package mBeans;
2
3 import java.util.concurrent.atomic.AtomicLong;
4
5 public class HitPercentage implements HitPercentageMBean { 3 usages
6     private final AtomicLong totalCount = new AtomicLong(initialValue: 0); 3 usages
7     private final AtomicLong hitCount = new AtomicLong(initialValue: 0); 3 usages
8
9     @Override no usages
10    public double getHitPercentage() {
11        long total = totalCount.get();
12        if (total == 0) {
13            return 0.0;
14        } else {
15            return (double) hitCount.get() / total * 100.0;
16        }
17    }
18
19    public void addPoint(boolean hit) { 1 usage
20        totalCount.incrementAndGet();
21        if (hit) hitCount.incrementAndGet();
22    }
23
24    public void setInitialValues(long total, long hits) { 1 usage
25        totalCount.set(total);
26        hitCount.set(hits);
27    }
28 }
29
```

Рис 4. – HitPercentage.

PointsCounterMBean:

```
1 package mBeans;
2
3 public interface PointsCounterMBean { no usages 1 implementation
4     long getTotalPoints(); no usages 1 implementation
5     long getMissedPoints(); no usages 1 implementation
6 }
7
```

Рис 5. – PointsCounterMBean.

PointsCounter:

```
1 package mBeans;
2
3 import javax.management.Notification;
4 import javax.management.NotificationBroadcasterSupport;
5 import java.util.concurrent.atomic.AtomicLong;
6
7 public class PointsCounter extends NotificationBroadcasterSupport implements PointsCounterMBean { 3 usages
8     private final AtomicLong totalPoints = new AtomicLong( initialValue: 0); 3 usages
9     private final AtomicLong missedPoints = new AtomicLong( initialValue: 0); 3 usages
10    private int consecutiveMisses = 0; 3 usages
11    private long sequenceNumber = 1; 1 usage
12
13    @Override no usages
14    public long getTotalPoints() { return totalPoints.get(); }
15
16    @Override no usages
17    public long getMissedPoints() { return missedPoints.get(); }
18
19    public void addPoint(boolean hit) { 1 usage
20        totalPoints.incrementAndGet();
21        if (!hit) {
22            missedPoints.incrementAndGet();
23            consecutiveMisses++;
24            if (consecutiveMisses >= 4) {
25                sendNotification(new Notification(
26                    type: "consecutive.misses", source: this, sequenceNumber++,
27                    System.currentTimeMillis(), message: "пользователь промахнулся 4 раза подряд"
28                ));
29            }
30        } else {
31            consecutiveMisses = 0;
32        }
33    }
34
35    public void setInitialValues(long total, long missed) { 1 usage
36        totalPoints.set(total);
37        missedPoints.set(missed);
38    }
```

Рис 6. – PointsCounter.

MBeanRegistry:

```
1 package mBeans;
2
3 > import ...
12
13 @ApplicationScoped 2 usages
14 public class MBeanRegistry {
15
16     @Inject 1 usage
17     private DatabaseManager databaseManager;
18
19     private final PointsCounter pointsCounter = new PointsCounter(); 3 usages
20     private final HitPercentage hitPercentage = new HitPercentage(); 3 usages
21
22     @PostConstruct
23     public void init() {
24         try {
25             List<HitEntity> allResults = databaseManager.getAllResults();
26             long total = allResults.size();
27             long missed = allResults.stream().filter( HitEntity h -> !h.isHit()).count();
28             long hits = total - missed;
29
30             pointsCounter.setInitialValues(total, missed);
31             hitPercentage.setInitialValues(total, hits);
32
33             MBeanServer mbs = ManagementFactory.getPlatformMBeanServer();
34             mbs.registerMBean(pointsCounter, new ObjectName("Lab3:type=PointsCounter"));
35             mbs.registerMBean(hitPercentage, new ObjectName("Lab3:type=HitPercentage"));
36         } catch (Exception e) {
37             e.printStackTrace();
38         }
39     }
40
41     public PointsCounter getPointsCounter() { return pointsCounter; } 1 usage
42     public HitPercentage getHitPercentage() { return hitPercentage; } 1 usage
43 }
```

Рис 7. – MBeanRegistry.

Обновление бинов в классе PointDataBean:

```

15 > @{...}
17 public class PointDataBean implements Serializable {
18     private static final long serialVersionUID = 1L; no usages
19
20     private BigDecimal x; 4 usages
21     private BigDecimal y; 4 usages
22     private BigDecimal r; 4 usages
23     private boolean hit; 6 usages
24
25     @Inject 2 usages
26     private MBeanRegistry mBeanRegistry;
27
28     @Inject 1 usage
29     private HitHistoryBean hitHistoryBean;
30
31     @Inject 1 usage
32     private DatabaseManager databaseManager;
33
34     @PostConstruct
35     public void init() {...}
36
37
38
39
40
41     public void check() { 4 usages
42
43         hit = CheckArea.checkHit(x, y, r);
44         HitEntity hitEntity = new HitEntity(x, y, r, hit);
45         databaseManager.save(hitEntity);
46         hitHistoryBean.addHit(hitEntity);
47
48         mBeanRegistry.getPointsCounter().addPoint(hit);
49         mBeanRegistry.getHitPercentage().addPoint(hit);
50     }

```

Рис 8. – PointDataBean.

Задание 2.

Показание MBeans, разработанных в ходе задания 1:

HitPercentage:

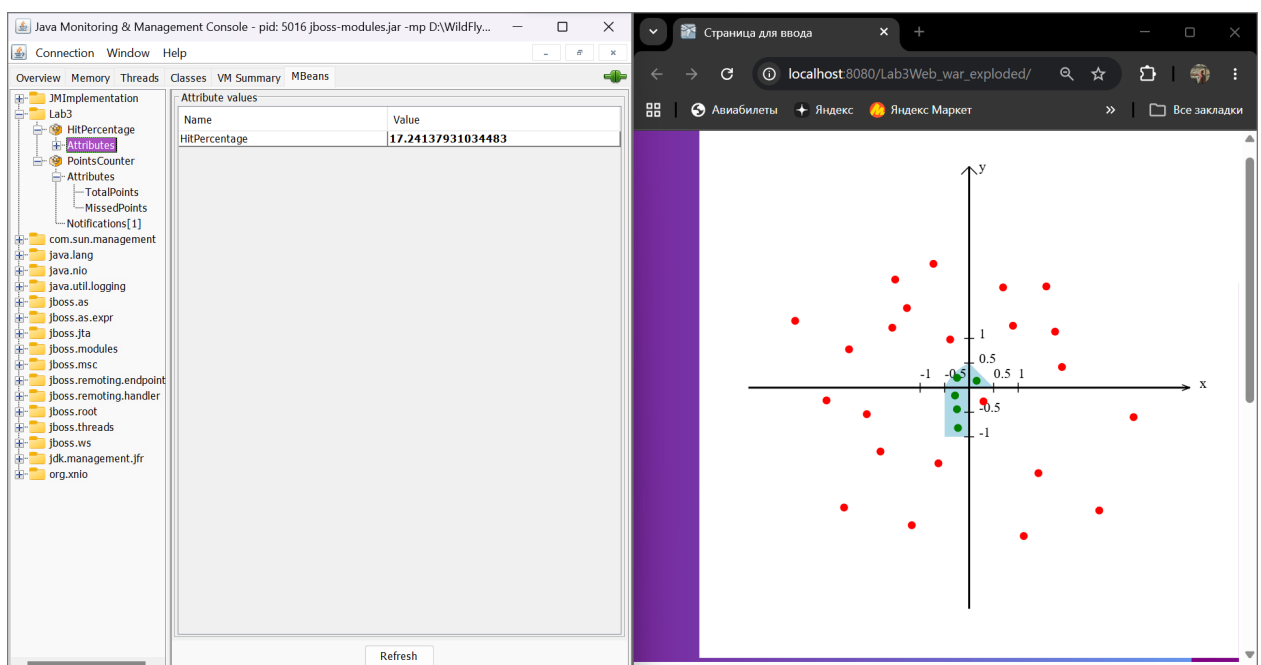


Рис 9.

PointsCounter:

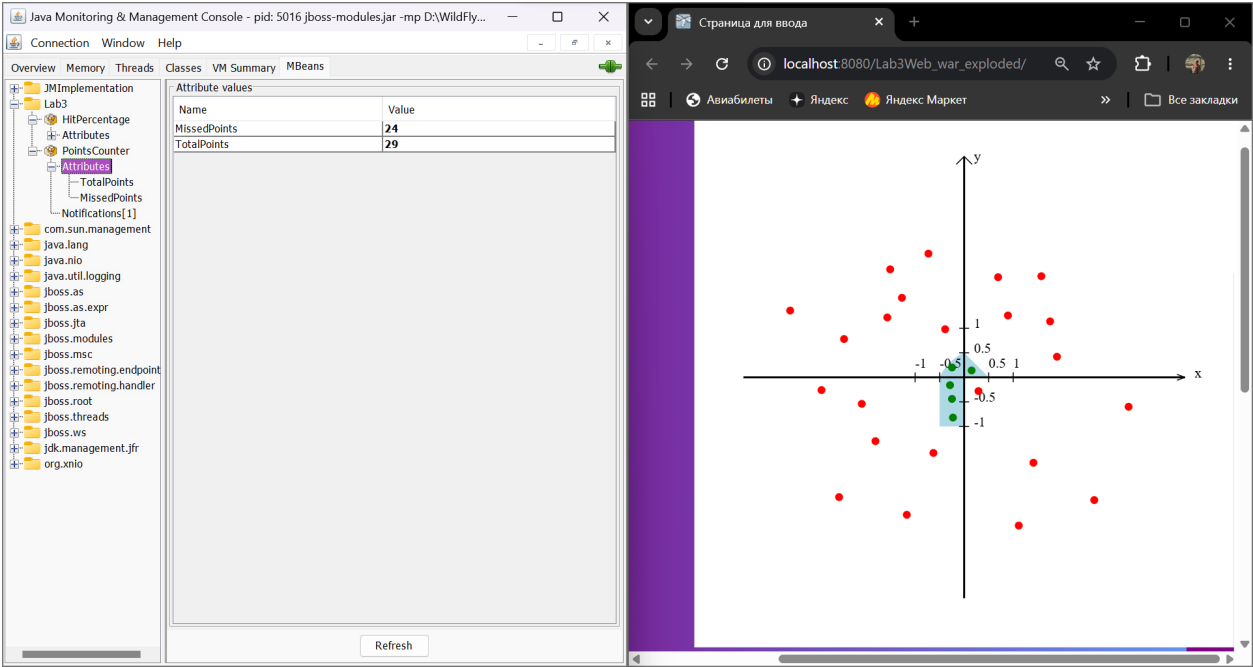


Рис 10.

Уведомления из PointsCounter:

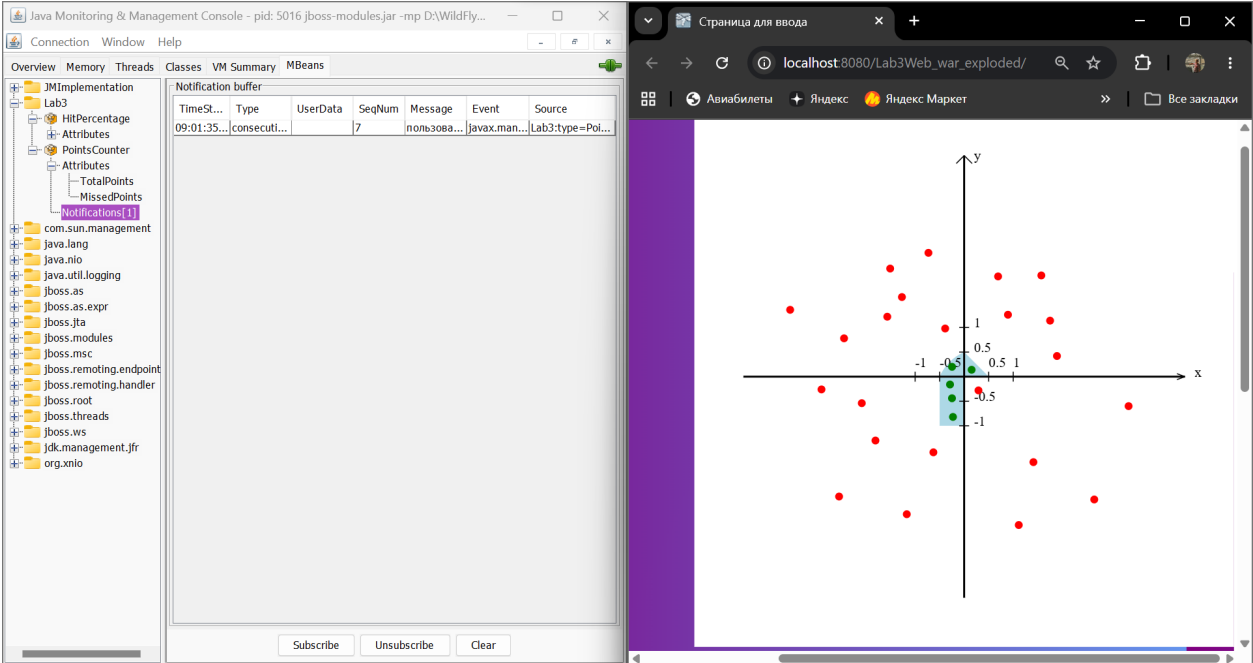


Рис 11.

Имя и версия ОС:

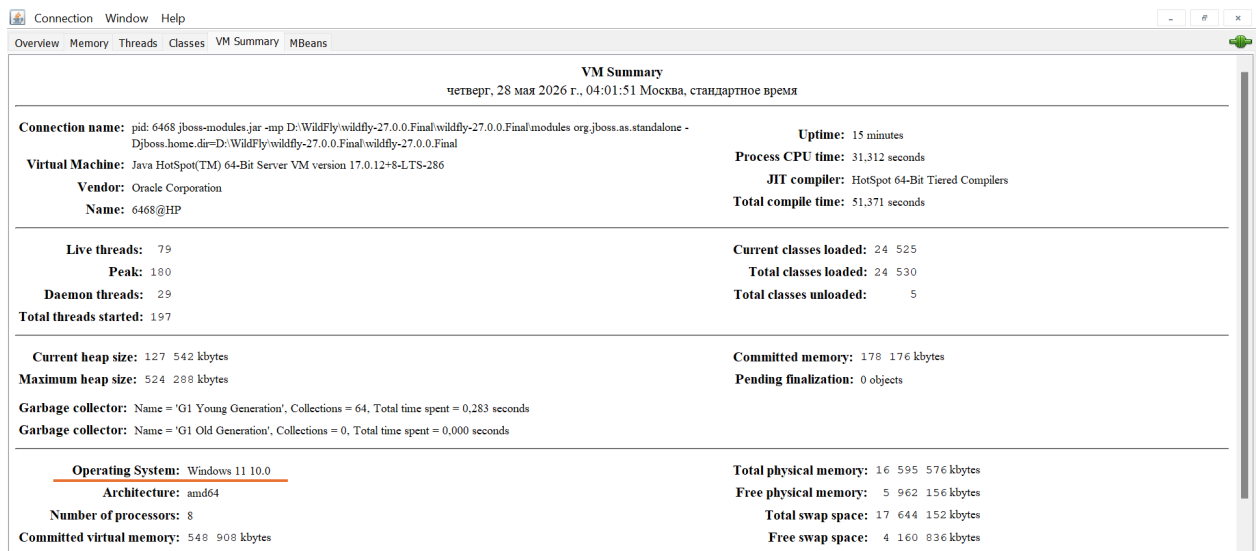


Рис 12.

Выводы по заданию 2:

С помощью утилиты JConsole были проанализированы кастомные управляемые бины, разработанные в первой части задания. В процессе мониторинга были получены текущие значения их атрибутов (TotalPoints, MissedPoints, HitPercentage) и проверена работа уведомлений. Была определена среда выполнения JVM, программа запущена под управлением ОС Windows 11 10.0

Задание 3.

График изменения показаний MBean-классов:

HitPercentage:

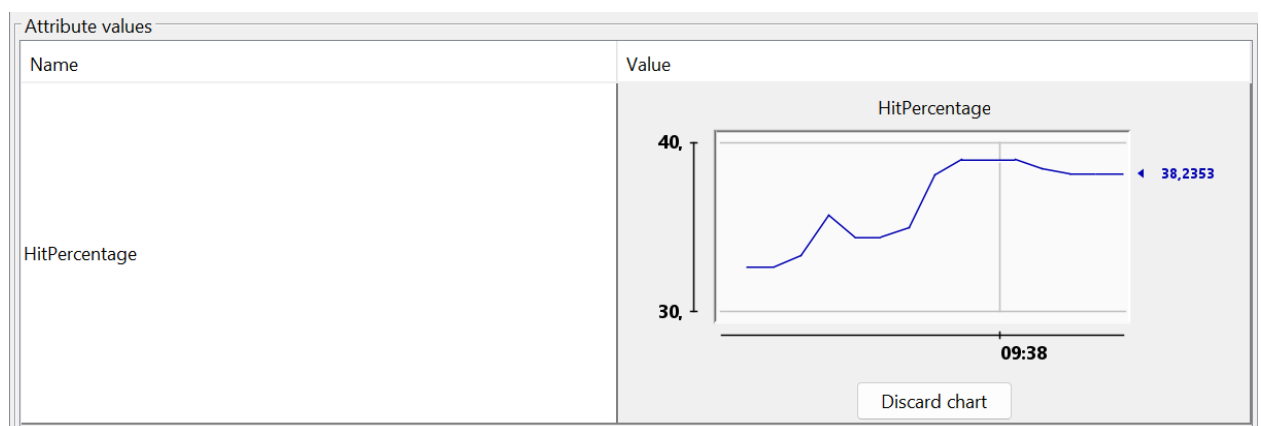


Рис 13.

PointsCounter:

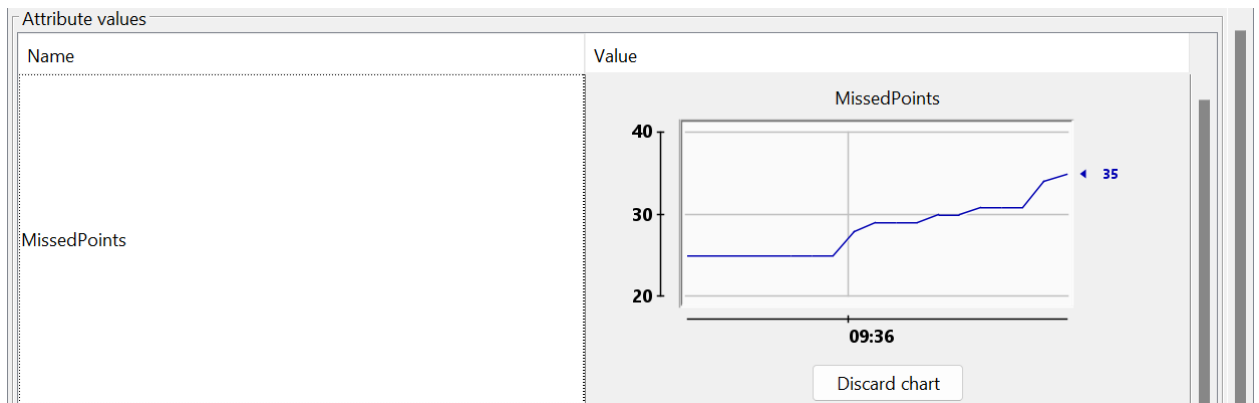


Рис 14.

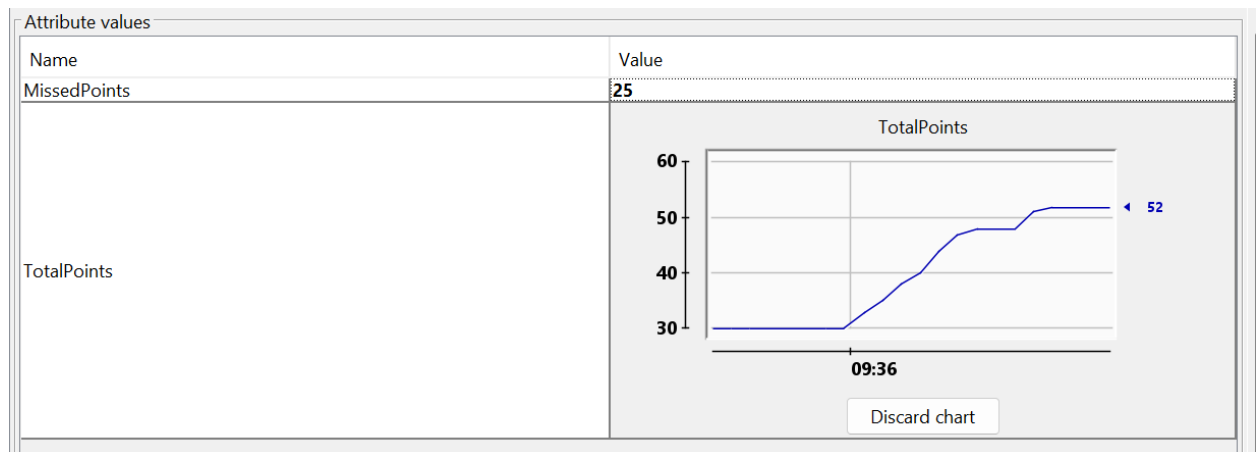


Рис 15.

Имя потока, потребляющего наибольший процент времени CPU:

org.jboss.modules.Main (pid 16484)

Sampler Settings

Samp... CPU Memory Stop

Status: CPU sampling in progress

CPU samples Thread CPU time

Results: View: Collected data: Snapshot

Name	Total Time	Total Time (CPU)
RMI TCP Connection(1)-172.18.0.1	32 358 ms (100 %)	32 358 ms (100 %)
RMI TCP Connection(idle)	29 220 ms (100 %)	29 220 ms (100 %)
default task-1	61 496 ms (100 %)	2 073 ms (100 %)
default I/O-12	61 496 ms (100 %)	271 ms (100 %)
DeploymentScanner-threads - 1	61 496 ms (100 %)	92,1 ms (100 %)
default I/O-1	61 496 ms (100 %)	77,5 ms (100 %)
Reference Reaper	61 496 ms (100 %)	0,0 ms (- %)
MSC service thread 1-1	61 496 ms (100 %)	0,0 ms (- %)
MSC service thread 1-2	61 496 ms (100 %)	0,0 ms (- %)
MSC service thread 1-3	61 496 ms (100 %)	0,0 ms (- %)
MSC service thread 1-4	61 496 ms (100 %)	0,0 ms (- %)
MSC service thread 1-5	61 496 ms (100 %)	0,0 ms (- %)
MSC service thread 1-6	61 496 ms (100 %)	0,0 ms (- %)
MSC service thread 1-7	61 496 ms (100 %)	0,0 ms (- %)
MSC service thread 1-8	61 496 ms (100 %)	0,0 ms (- %)

Рис 16.

Имя потока - RMI TCP Connection(1)-172.18.0.1

Выводы по заданию:

В ходе выполнения задания во вкладке “MBeans” были выбраны атрибуты MBeans, разработанных в задании 1. Были получены графики в реальном времени изменения.

С помощью инструмента CPU Sampler был проведен анализ времени работы потоков. В результате профилирования было определено, что наибольший процент времени CPU потребляет поток с именем RMI TCP Connection(1)-172.18.0.1.

Задание 4.

Проблемы с производительностью в программе:

Проблема 1:

В ходе анализа работы приложения было замечено, что программа работает медленно и неэффективно использует ресурсы процессора. При профилировании CPU в VisualVM обнаружено, что основной поток main практически всё время находится в состоянии ожидания. Анализ дерева вызовов показывает, что метод Main.main() вызывает java.lang.Thread.sleep() на каждой итерации цикла.

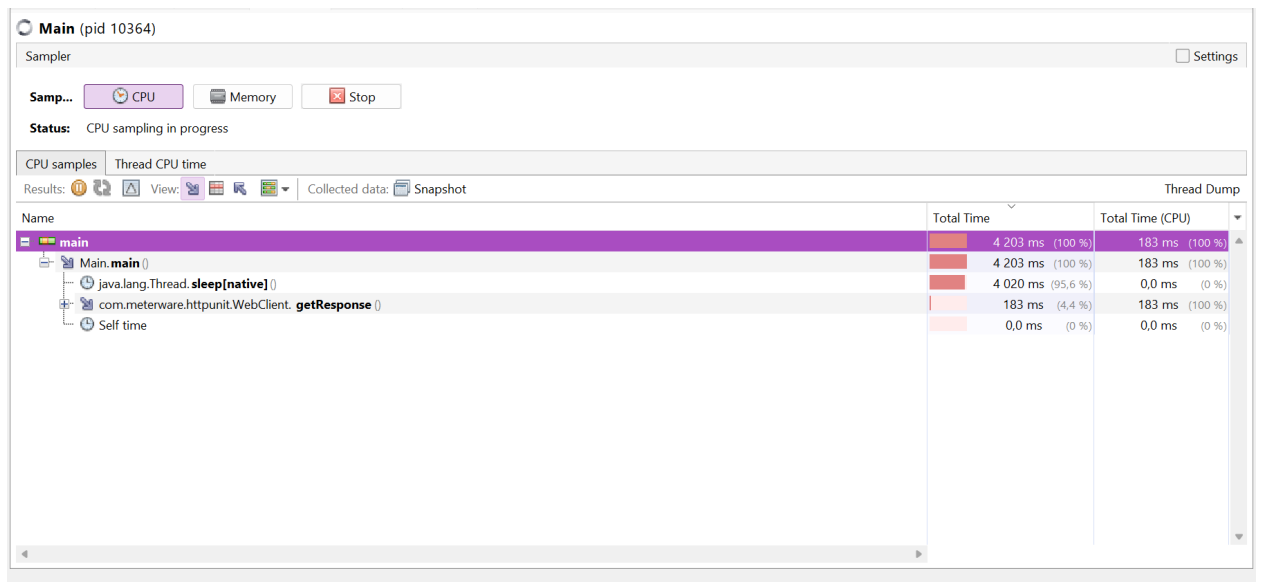


Рис 17.

```

public static void main(String[] args) {
    try {
        HttpUnitOptions.setExceptionsThrownOnScriptError(false);
        ServletRunner sr = new ServletRunner();
        sr.registerServlet( resourceName: "myServlet", HelloWorld.class.getName());
        ServletUnitClient sc = sr.newClient();
        int number = 1;
        WebRequest request = new GetMethodWebRequest( urlString: "http://test.meterware.com/myServlet");
        while (true) {
            WebResponse response = sc.getResponse(request);
            System.out.println("Count: " + number++ + response);
            java.lang.Thread.sleep( millis: 200);
        }
    } catch (InterruptedException ex) {
        Logger.getLogger( name: "global").log(Level.SEVERE, msg: null, ex);
    } catch (MalformedURLException ex) {
        Logger.getLogger( name: "global").log(Level.SEVERE, msg: null, ex);
    } catch (IOException ex) {
        Logger.getLogger( name: "global").log(Level.SEVERE, msg: null, ex);
    } catch (SAXException ex) {
        Logger.getLogger( name: "global").log(Level.SEVERE, msg: null, ex);
    }
}

```

Рис 18.

Данная операция не несла полезной нагрузки, поэтому для решения данной проблемы было решено закомментировать данные строки:

```

public static void main(String[] args) {
    try {
        HttpUnitOptions.setExceptionsThrownOnScriptError(false);
        ServletRunner sr = new ServletRunner();
        sr.registerServlet( resourceName: "myServlet", HelloWorld.class.getName());
        ServletUnitClient sc = sr.newClient();
        int number = 1;
        WebRequest request = new GetMethodWebRequest( urlString: "http://test.meterware.com/myServlet");
        while (true) {
            WebResponse response = sc.getResponse(request);
            System.out.println("Count: " + number++ + response);
            //java.lang.Thread.sleep(200);
        }
    } catch (InterruptedException ex) {
        //Logger.getLogger("global").log(Level.SEVERE, null, ex);
    } catch (MalformedURLException ex) {
        Logger.getLogger( name: "global").log(Level.SEVERE, msg: null, ex);
    } catch (IOException ex) {
        Logger.getLogger( name: "global").log(Level.SEVERE, msg: null, ex);
    }
}

```

Рис 19.

Также это было сделано для более удобного решения проблемы 2.

Профилирование CPU в VisualVM после комментирования `java.lang.Thread.sleep(200);`:

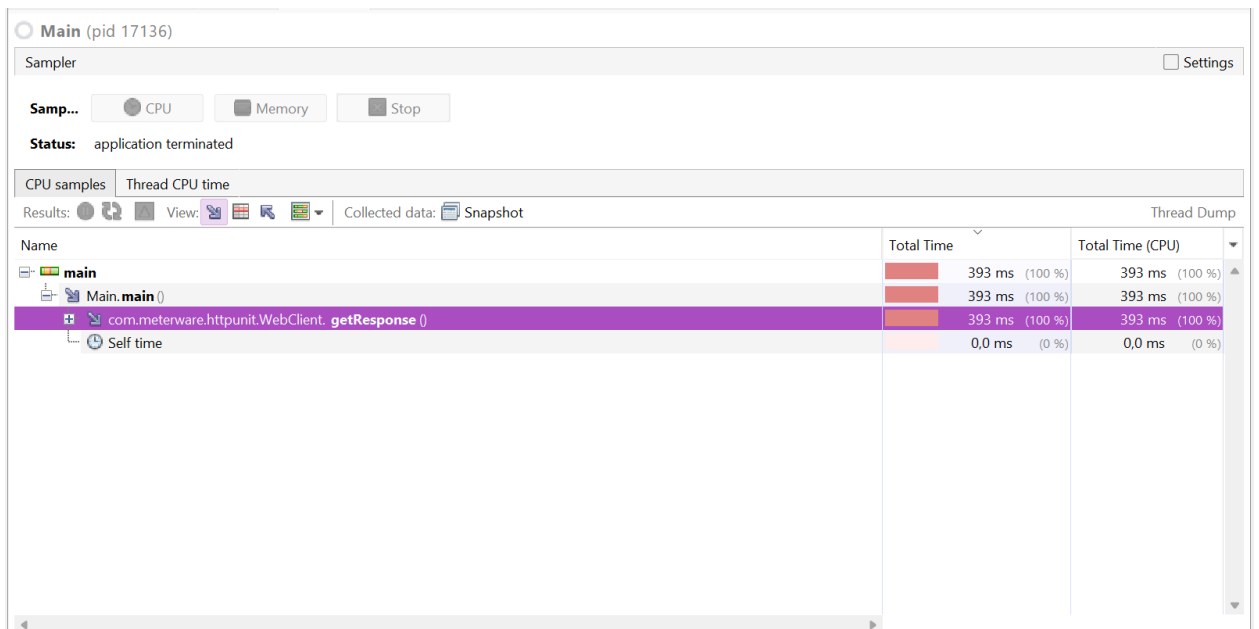


Рис 20.

Проблема 2:

После удаления задержки программа начала работать быстро, но через короткое время стала аварийно завершаться с ошибкой `java.lang.OutOfMemoryError: Java heap space`, программа завершается после 9 минут работы.

```
[java] Count: 2498[ _response = com.meterware.servletunit.ServletUnitHttpResponse@7b33deed]
[java] Count: 2499[ _response = com.meterware.servletunit.ServletUnitHttpResponse@7d75940]
[java] Count: 2500[ _response = com.meterware.servletunit.ServletUnitHttpResponse@2db6ba81]
[java] Count: 2501[ _response = com.meterware.servletunit.ServletUnitHttpResponse@3dc39412]
[java] Exception in thread "main" java.lang.OutOfMemoryError: Java heap space
[java]     at com.meterware.httpunit.WebWindow.getResponse(WebWindow.java:120)
[java]     at com.meterware.httpunit.WebClient.getResponse(WebClient.java:113)
[java]     at Main.main(Main.java:46)
```

Рис 21.

Проблема воспроизводится стабильно даже при ограничении кучи до 40 МБ

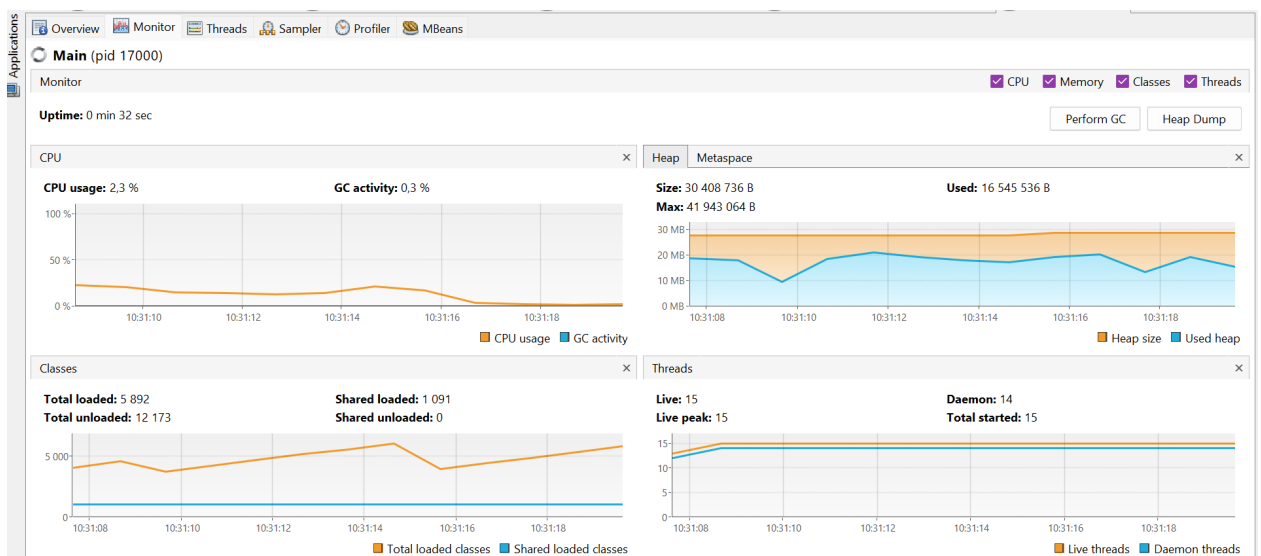


Рис 22.

В утилите VisualVM заходим во вкладку Monitor, видим график Heap, где синяя область (используемая память) неуклонно растет вверх.

В VisualVM открываем heap dump, во вкладке Summary и Objects видим большое количество объектов byte[] и String.

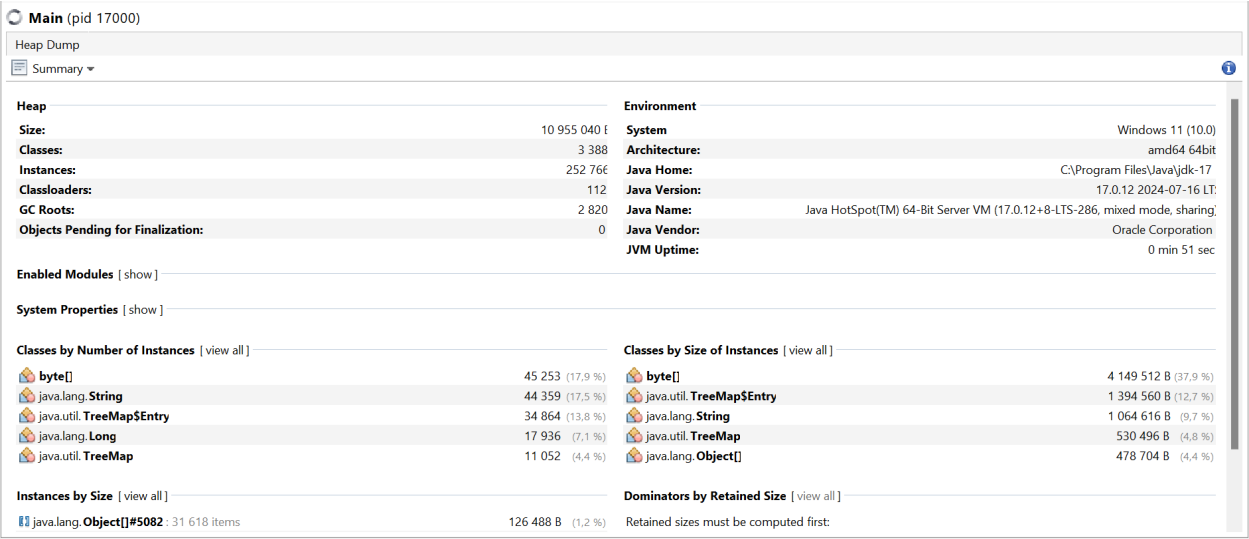


Рис 23.

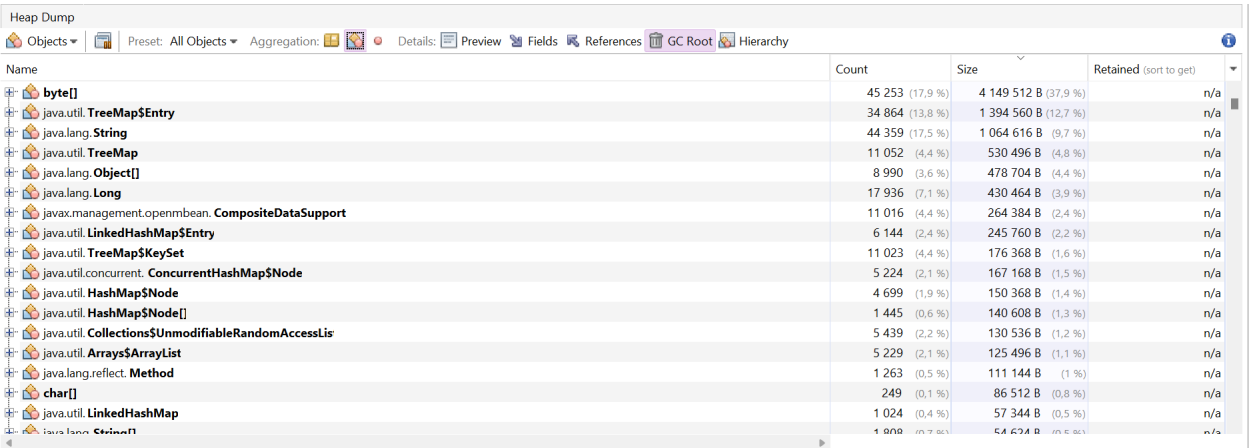


Рис 24.

Заходим в References во вкладке Instances для построения пути к корню сборщика мусора. Поскольку сборщик мусора не может удалить объект, на который существует ссылка, анализ цепочки ссылок является единственным способом определить, какой именно программный узел препятствует освобождению памяти.

Name	Size	Retained (sort to get)
java.lang.Object[] #5082 : 31 618 items	126 488 B (1.2 %)	n/a
<items>		
<references>		
elementData in java.util.ArrayList#947 : 23 838 elements	24 B (0 %)	n/a
static _errorMessages in class com.meterware.httpunit.javascript.JavaScript : JavaScript	72 B (0 %)	n/a
[51] in java.lang.Object[] #5080 : 549 items	2 216 B (0 %)	n/a

Рис 25.

Было обнаружено, что утечка происходит из-за статического списка, в который HttpUnit записывает ошибки JavaScript. Причиной ошибок стала опечатка в сервлете HelloWorld.java. В бесконечном цикле этот список бесконечно разрастался.

Устранение проблемы:

1.

Находим в коде класс `com.meterware.httpunit.javascript.JavaScript`, а в нём и метод для очистки сообщений об ошибках.

```

58 |
59 | > static void clearErrorMessages() { _errorMessages.clear(); }
62 |

```

Рис 26.

Чтобы не менять сигнатуру метода и спокойно вызвать его из `main` продолжаем искать в коде какую-нибудь обёртку для этого метода.

Находим метод `clearErrorMessages` в классе `com.meterware.httpunit.javascript.JavaScriptEngineFactory`

```

82 |
83 | > public void clearErrorMessages() { JavaScript.clearErrorMessages(); }
86 |
87 |

```

Рис 27.

И после находим метод `clearScriptErrorMessages` в классе `com.meterware.httpunit.HttpUnitOptions`, который мы уже можем вызвать из `main`.

```

public Main() { 2 usages
}

/**
 * @param args the command line arguments
 */
public static void main(String[] args) {
    try {
        HttpUnitOptions.setExceptionsThrownOnScriptError(false);
        ServletRunner sr = new ServletRunner();
        sr.registerServlet(resourceName: "myServlet", HelloWorld.class.getName());
        ServletUnitClient sc = sr.newClient();
        int number = 1;
        WebRequest request = new GetMethodWebRequest(urlString: "http://test.meterware.com/myServlet");
        while (true) {
            com.meterware.httpunit.HttpUnitOptions.clearScriptErrorMessages();

            WebResponse response = sc.getResponse(request);
            System.out.println("Count: " + number++ + response);
        }
    }
}

```

Рис 29.

На каждой итерации Main вызываем этот метод, чтобы очистить сообщения об ошибках.

2.

```

23      @Override no usages
24      public void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
25          PrintWriter out = response.getWriter();
26          response.setContentType("text/html");
27          out.println("<!DOCTYPE HTML PUBLIC \"-//W3C//DTD HTML 4.0 \" +
28                      \"Transitional//EN\">\n" +
29                      "<HTML>\n" +
30                      "<HEAD><TITLE>Hello World</TITLE></HEAD>\n" +
31                      "<BODY>\n" +
32                      "<H1>Hello World</H1>\n");
33          out.println("<script language=\"JavaScript\" type=\"text/javascript\">");
34          out.println("<!--");
35          out.println("document.write('Hello Document')");
36          out.println("//-->");
37          out.println("</script>");
38          out.println("</BODY></HTML>");
39      }
40  }
41

```

Рис 30.

Убираем опечатку (с document.wr на document.write), чтобы устранить причину ошибки.

После этого программа работает исправно.

```

[java] Count: 16310[ _response = com.meterware.servletunit.ServletUnitHttpResponse@61af1a9c]
[java] Count: 16311[ _response = com.meterware.servletunit.ServletUnitHttpResponse@49a40d39]
[java] Count: 16312[ _response = com.meterware.servletunit.ServletUnitHttpResponse@4bca65ec]
[java] Count: 16313[ _response = com.meterware.servletunit.ServletUnitHttpResponse@106111b1]
[java] Count: 16314[ _response = com.meterware.servletunit.ServletUnitHttpResponse@2e63c30f]
[java] Count: 16315[ _response = com.meterware.servletunit.ServletUnitHttpResponse@3eee59ba]
[java] Count: 16316[ _response = com.meterware.servletunit.ServletUnitHttpResponse@346114215]
[java] Count: 16317[ _response = com.meterware.servletunit.ServletUnitHttpResponse@2579fa8f]
[java] Count: 16318[ _response = com.meterware.servletunit.ServletUnitHttpResponse@37ae958]
[java] Count: 16319[ _response = com.meterware.servletunit.ServletUnitHttpResponse@5c665212]
[java] Count: 16320[ _response = com.meterware.servletunit.ServletUnitHttpResponse@38ec69da]
[java] Count: 16321[ _response = com.meterware.servletunit.ServletUnitHttpResponse@34570603]
[java] Count: 16322[ _response = com.meterware.servletunit.ServletUnitHttpResponse@3cc65e0f]
[java] Count: 16323[ _response = com.meterware.servletunit.ServletUnitHttpResponse@5978b015]
[java] Count: 16324[ _response = com.meterware.servletunit.ServletUnitHttpResponse@44666645]
[java] Count: 16325[ _response = com.meterware.servletunit.ServletUnitHttpResponse@2a93bb5e]
[java] Count: 16326[ _response = com.meterware.servletunit.ServletUnitHttpResponse@4a6950d1]
[java] Count: 16327[ _response = com.meterware.servletunit.ServletUnitHttpResponse@41df2efe]
[java] Count: 16328[ _response = com.meterware.servletunit.ServletUnitHttpResponse@30d41ce]
[java] Count: 16329[ _response = com.meterware.servletunit.ServletUnitHttpResponse@74fa79c4]
[java] Count: 16330[ _response = com.meterware.servletunit.ServletUnitHttpResponse@3b926842]

```

Рис 31.

IntelliJ Profiler:

Запуск в IntelliJ при максимальном размере heap в 12 МБ.

```

Count: 29085[ _response = com.meterware.servletunit.ServletUnitHttpResponse@2783ca4e]
Count: 29086[ _response = com.meterware.servletunit.ServletUnitHttpResponse@2b7ce044]
Exception in thread "main" java.lang.OutOfMemoryError: GC overhead limit exceeded
    at java.lang.Class.getDeclaredMethods0(Native Method)
    at java.lang.Class.privateGetDeclaredMethods(Class.java:2720)
    at java.lang.Class.privateGetMethodRecursive(Class.java:3076)
    at java.lang.Class.getMethod0(Class.java:3044)
    at java.lang.Class.privateGetMethodRecursive(Class.java:3086)
    at java.lang.Class.getMethod0(Class.java:3044)
    at java.lang.Class.getMethod(Class.java:1812)
    at org.cyberneko.html.HTMLConfiguration.reset(Unknown Source)
    at org.cyberneko.html.HTMLConfiguration.setInputSource(Unknown Source)
    at org.cyberneko.html.HTMLConfiguration.parse(Unknown Source)
    at org.apache.xerces.parsers.XMLParser.parse(Unknown Source)
    at org.apache.xerces.parsers.DOMParser.parse(Unknown Source)
    at com.meterware.httpunit.parsing.NekoHTMLParser.parse(NekoHTMLParser.java:41)
    at com.meterware.httpunit.HTMLPage.parse(HTMLPage.java:255)
    at com.meterware.httpunit.WebResponse.getReceivedPage(WebResponse.java:1109)
    at com.meterware.httpunit.WebResponse.getFrames(WebResponse.java:1098)
    at com.meterware.httpunit.WebResponse.getFrameRequests(WebResponse.java:875)
    at com.meterware.httpunit.FrameHolder.updateFrames(FrameHolder.java:179)
    at com.meterware.httpunit.WebWindow.updateFrameContents(WebWindow.java:252)
    at com.meterware.httpunit.WebClient.updateFrameContents(WebClient.java:485)
    at com.meterware.httpunit.WebWindow.updateWindow(WebWindow.java:146)
    at com.meterware.httpunit.WebWindow.getSubframeResponse(WebWindow.java:130)
    at com.meterware.httpunit.WebWindow.getResponse(WebWindow.java:121)
    at com.meterware.httpunit.WebClient.getResponse(WebClient.java:113)
    at Main.main(Main.java:49)

Process finished with exit code 1

```

Рис 32.

Причиной переполнения списка стала опечатка в сервлете HelloWorld.java: вместо стандартного метода `document.write` использовался несуществующий `document.wr`. Поскольку вызов происходил внутри бесконечного цикла, библиотека `HttpUnit` непрерывно записывала сообщения об ошибках и сохраняла их в статическую коллекцию, что привело к исчерпанию доступной памяти.

Для решения проблемы в основной цикл `main` был добавлен вызов `HttpUnitOptions.clearScriptErrorMessages()`, что гарантирует очистку буфера ошибок на каждой итерации и стабильное потребление памяти. Опечатка в коде сервлета была исправлена.

Вывод по лабораторной работе:

В ходе выполнения лабораторной работы я получил навыки мониторинга и оптимизации Java-приложений с использованием `VisualVM` и встроенных профилировщиков IDE. Работа с `MBean`-компонентами и инструментами анализа ресурсов позволила в реальном времени отслеживать состояние JVM.

Ключевой частью стала локализация утечки памяти, использование `Heap Dump` и анализ цепочек ссылок до корней `GC Roots` позволили обнаружить неявную проблему - неконтрольный рост статической коллекции внутри программы.

В результате проделанной работы была достигнута оптимальная производительность системы.